

欢迎来到 C++ 的魔法世界！——用 VSCode 开启你的第一段代码冒险

👋 亲爱的编程萌新们：

你们即将踏上一段神奇的旅程：用键盘写代码，让电脑听你指挥！

而我们的“魔法杖”就是——**Visual Studio Code (简称 VSCode)**。

别慌，它不难，而且超级酷！

✅ 任务一：搭建 C++ 开发环境

使用 **Visual Studio Code** 与 **MinGW-w64** 搭建轻量级 C++ 开发环境，实现以下功能：

1. VSCode 是谁？它和“编译器”是什么关系？

想象一下：

- **VSCode** 就像你的“笔记本”或“写字板”——你在这里写代码，但它本身**不会运行代码**。
- **编译器**（比如 `g++`）才是真正的“翻译官”——它把你看得懂的 C++ 代码，翻译成电脑能懂的“机器语言”。

🎯 所以：

VSCode 写代码 📝 → 编译器翻译成 exe 📦 → 电脑运行程序 🚀

💡 VSCode 的三大优点

1. **轻量又快**：不像某些“重量级选手”（比如 Visual Studio），VSCode 启动飞快，像小电驴，不占内存！
2. **插件超多**：装个插件，它就能写 Python、Java、HTML.....甚至帮你画流程图！堪称“代码界的瑞士军刀”。
3. **跨平台**：Windows、Mac、Linux 都能用，以后换电脑也不怕！

🔧 2. 编译器小课堂：MinGW vs MSVC

你可能会听说两个名字：**MinGW** 和 **MSVC**，它们都是“翻译官”，但来自不同家族：

名字	全称	特点
MinGW	Minimalist GNU for Windows	开源、免费、轻量，
MSVC	Microsoft Visual C++	微软亲儿子，功能强，但有点重

🎯 下面我们以 **MinGW** 为例，因为它免费、简单、够用！当然我们也欢迎你尝试 **MSVC**！

🌸 3. 程序是怎么跑起来的？三步走

每一段 C++ 代码，都要经历“三重考验”才能变成可运行的程序：

1. 编译 (Compile)

→ 把 `.cpp` 文件翻译成“中间语言”（就像草稿翻译成摩斯密码）。

2. 链接 (Link)

→ 如果你写了多个文件，要把它们“拼”在一起（就像拼乐高）。

3. 生成 .exe 文件

→ 得到一个可以双击运行的程序！🎉

📌 简单记：

写代码 → 编译 → 链接 → 出 exe → 运行！

🔧 4. 你的任务：自己动手，丰衣足食

你需要完成以下几步：

1. 下载并安装 VSCode

2. 下载并配置 MinGW 编译器

3. 在 VSCode 中安装插件

4. 配置编译和运行环境

🌸 5. 验证你的开发环境是否成功

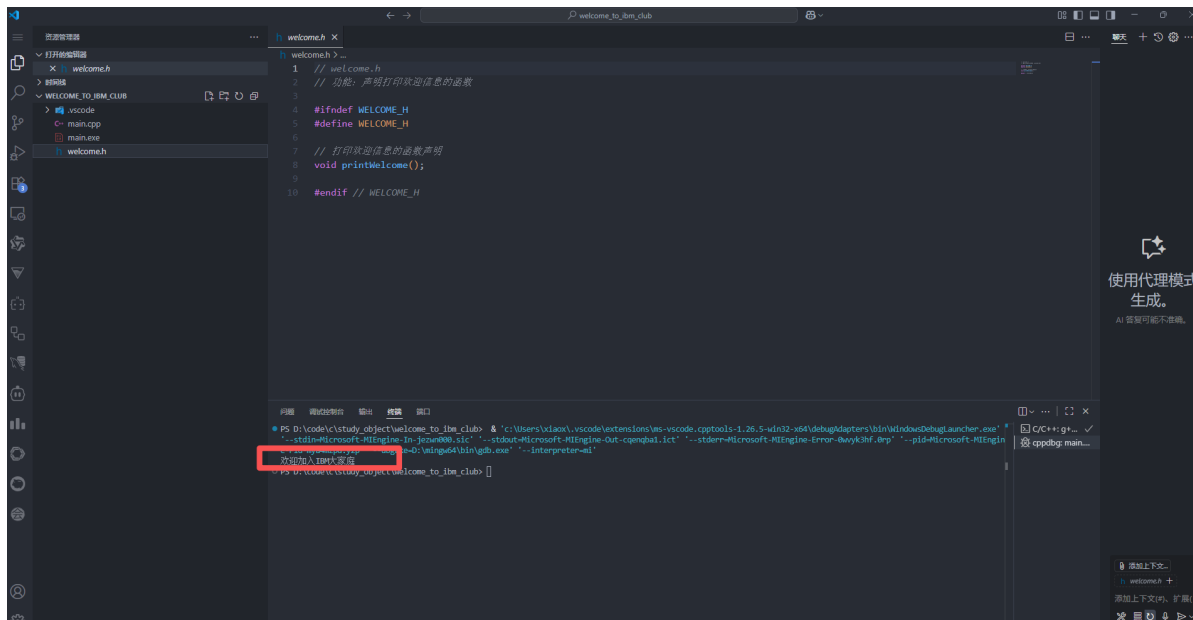
如果你已经配置好 VSCode + MinGW 环境，可以用一个简单的多文件程序来测试是否能正常编译和运行。（代码文件我已经一起打包给你啦！）

🎉 6. 联系方式及提交

负责人微信号：x846637 (记得带备注添加，学姐遇到陌生人一般直接略过)

请提交你的运行结果到负责人微信

示例图片如下(如果看不清的话，也可以在文件夹里直接打开这个图片)，记得展示你的运行结果！



✅ 任务二：实现OOP编程实验

任务二旨在帮助你熟悉面向对象编程（Object Oriented Programming）的思想以及模式。使用VSCode（也可以使用其他你觉得顺手的编程软件）完成OOP编程实验。

😬 1.什么是OOP呢？

以下是某百科给出的定义：

面向对象程序设计（Object Oriented Programming，OOP）是一种计算机编程架构。OOP的一条基本原则是计算机程序由单个能够起到子程序作用的单元或对象组合而成。OOP达到了软件工程的三个主要目标：重用性、灵活性和扩展性。OOP=对象+类+继承+多态+消息，其中核心概念是类和对象。



通俗一点就是：当解决一个问题的时候，面向对象会把事物抽象成对象的概念，就是说这个问题里面有哪些对象，然后赋予对象一些属性和方法，让每个对象去执行自己的方法，使问题得到解决。

OOP的核心概念：封装、继承、多态和抽象

😬 2.OOP有什么特点呢？

- **模块化：** OOP鼓励将大型问题分解为更小、更易于管理的部分，每个部分都是一个独立的对象。这样，开发人员可以专注于单个对象的设计和实现，而不需要担心整个系统的细节
- **代码重用：** 通过继承机制，新类可以从现有类中继承属性和方法，避免了重复编写相同的代码，节省了开发时间，降低了出错的概率。
- **易于维护** 由于对象封装了状态和行为，所以修改一个对象的内部实现不会影响其他对象，只要对外接口不变。这样，当需要更新或修复代码时，可以更加容易地进行维护。
- **提高代码质量** 面向对象的设计原则（如单一职责原则、开闭原则等）鼓励写出更加清晰、可读性强、易于理解的代码，从而提高整体代码的质量。

📌 3.你需要干什么呢：大胆尝试吧！

以学校教务系统中的成员信息举例。人的信息作为一个基础模板，而学生的信息作为继承了人特性的另一种类，既包含了人共通的信息（如：姓名），又有自己独有的信息（如：学号）。在这里，我们可以认为人是基类，学生是人的一个派生类。

👉 在这个任务里面，你可以根据上面信息为背景对类进行编写，也可以自己设计背景并编写自己独特的类和对象。

📌 要求：

所编写的类**必须要体现封装和继承的特性**，最好可以体现出“继承”的关系，并且在主函数中创建类的对象并调用类的方法进行演示，以此来验证事件逻辑的完整性。

本任务只需要完成面向对象编程的基础部分，**不对事件逻辑的复杂度进行严格考核。**

4.联系方式及提交

两个任务的负责人不是同一个哦

vx: 3024602247 (记得带上备注哦, 学姐遇到陌生人一般直接忽略)

把任务源代码提交到负责人微信, 只需提交源代码!!!

任务三: Git分支和协作

Git是什么? 为什么要学习Git的使用?

Git是一个分布式版本控制系统, 由Linus Torvalds在2005年创建, 主要用于Linux内核的开发。它允许许多人协作开发同一个项目, 并且能够追踪每一个文件的每一次修改历史, 这样就能知道谁改了什么内容, 什么时候改的。此外, Git还能非常高效地处理从小型到非常大型的项目。

学习Git的原因有以下几点:

1. 版本控制: Git可以帮助你管理项目的各个版本, 方便你在需要的时候回到之前的版本状态。
2. 协作能力: 对于团队开发来说, Git提供了一个强大的框架来支持多个人同时在一个项目上工作。通过分支 (branch) 功能, 每个开发者可以创建自己的分支, 在不影响主分支的情况下进行开发, 开发完成后可以合并 (merge) 回主分支。
3. 备份: 由于Git是分布式的, 每个开发者本地都有一个完整的仓库副本, 这意味着即使服务器上的仓库损坏或丢失, 也可以从任何一个开发者的本地仓库恢复数据。

...

Git是现代软件开发中不可或缺的一项工具, 无论你是独立开发者还是团队的一员, 学习并掌握Git都将对你的职业生涯大有裨益。

Github上的开源资源非常丰富, 可以找到很多的学习资料以及各位以后可能会用到的各类技术资源。大家也可以在Github上传自己的心得, 资料等。还有更多的作用就等待同学们自己去探索了。

步骤概述:

创建和初始化Git仓库

- 创建一个github账号 (这一步可能需要魔法上网, 不会的同学可以使用Watt Toolkit软件或者自行搜索教程, 在Watt Toolkit软件的游戏加速选项中可以免费对于Github进行加速)。
- fork这个仓库: https://github.com/dolzhuying/IBM_test_round2

本地提交远程仓库

- 安装并配置Git (这里建议使用Git的纯命令行形式, 当然你也可以选择使用GUI界面)。
- 克隆上一步fork的仓库到本地。
- 创建一个新的分支, 以你的年级和姓名拼音命名。(这一步关系到最后的统计, 所以请保证无误, 为了方便统计请大家用大写字母分隔你姓名中的汉字, e.g. (e.g. 23LiuShengjie)
- 切换到你的新分支。
- 添加包含任务一和任务二的文件的文件夹到暂存区。
- 提交更改并且添加提交信息。
- 把本地的新分支推送到你的远程仓库。
- 在原始仓库 (https://github.com/dolzhuying/IBM_test_round2) 提出并请求pull request。

附加：撤销更改和回滚版本

如果有兴趣，你也可以附加以下操作并且将截图放入上一步的文件夹一起提交。

- 查看提交历史。
- 撤销本地的未提交更改。
- 创建新的提交以回滚到以前的版本。

附加题的用意在于让同学们了解到Git的强大作用并不止在于多人协作，更重要的是它是一个强大的版本管理工具，Git就像是一位细心的朋友，记得你每一步的改动，无论你是初学者还是老手，都能轻松找回过去的任何一个版本，再也不怕误操作。

注意事项：

！！！本项任务并不算难，同学们一开始可能会被陌生的名词吓到，但是这些工具的实际学习难度并不高，并且可以很简单地从网络上搜寻到相应的学习资料。

在学习过程中不用害怕犯错误，犯错也是成长的一部分。任务旨在带领大家学习一些以后常用的技术，并不会提出过高的要求，大家量力而行即可。如果不能完成任务三也可以添加前面任务的负责人微信提交前一/两个任务的结果，负责人会根据完成情况给予对应的考核分数

完成这些任务后，你会打开新世界的大门，拥有更广泛的技术知识和实践经验。